

Accenture OOP Preparation

Comprehensive Concept Notes & Practice Question Bank (Tier 1 & Tier 2)

TIER 1

1. What is OOP?

OOP (Object-Oriented Programming) is a programming paradigm in which programs are designed using objects that contain data and functions.

For example, a bank account can have:

Data:

- accountNumber
- balance
- ownerName

Functions:

- deposit()
- withdraw()
- checkBalance()

In C++:

```
class BankAccount {
public:
    int accountNumber;
    double balance;

    void deposit(double amount) {
        balance += amount;
    }
};
```

Here:

```
BankAccount
  ↓
  Class
  ↓
  Objects
```

Why OOP?

Main advantages:

- Code reusability
- Data security
- Modularity

- Maintainability
- Flexibility
- Easier representation of real-world entities

2. Class

A class is a blueprint/template for creating objects.

Example:

```
class Student {  
public:  
    string name;  
    int age;  
  
    void study() {  
        cout << "Studying";  
    }  
};
```

The class defines:

- Properties → `name` , `age`
- Behavior → `study()`

But the class itself is not a particular student.

3. Object

An object is an instance of a class.

```
Student s1;  
Student s2;
```

Here:

- `Student` → Class
- `s1` → Object
- `s2` → Object

They can contain different data:

```
s1.name = "Rahul";  
s2.name = "Amit";
```

★ Interview question

Q. What is the difference between class and object?

Answer: A class is a blueprint that defines properties and behavior, whereas an object is an instance of that class.

4. Encapsulation

Definition

Encapsulation is the bundling of data and methods into a single unit while restricting direct access to the internal data.

The unit is usually a class.

Example:

```
class BankAccount {  
private:  
    double balance;  
public:  
    void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
  
    double getBalance() {  
        return balance;  
    }  
};
```

Here:

```
balance
```

is private.

The user cannot directly modify it:

```
account.balance = -5000; // ❌
```

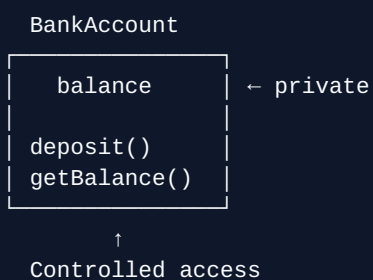
Instead:

```
account.deposit(5000); // ✅
```

Why? Because we can control how the data changes.

Encapsulation = Data Protection

Think:



Accenture trap

Don't say: "Encapsulation only means making variables private." That's incomplete.

Private members help achieve encapsulation, but encapsulation is broader: bundling state/behavior and controlling access to the object's internal state.

5. Abstraction

Definition

Abstraction means exposing only the essential functionality while hiding unnecessary implementation details.

Example: ATM.

You see:

- Insert card
- Enter PIN
- Withdraw money

You don't see:

- Database queries
- Bank-server communication
- Transaction validation
- Encryption

Those details are hidden.

Programming example

```
class Car {
public:
    void start() {
        checkEngine();
        injectFuel();
        ignite();
    }
private:
    void checkEngine() {
        // complex implementation
    }

    void injectFuel() {
        // complex implementation
    }

    void ignite() {
        // complex implementation
    }
};
```

The user simply calls:

```
car.start();
```

They don't need to know how the engine starts.

★ Encapsulation vs Abstraction

This is extremely important.

Encapsulation	Abstraction
Controls access to data/implementation	Hides unnecessary implementation complexity
Focuses on how data is protected	Focuses on what is exposed
Uses classes/access modifiers	Commonly uses abstract classes/interfaces
Data hiding is a major aspect	Implementation hiding is the main idea

Easy trick:

Encapsulation → Protect

Abstraction → Hide complexity

6. Inheritance

Definition

Inheritance allows a derived class to acquire properties and behaviors from a base class.

Example:

```
class Animal {
public:
    void eat() {
        cout << "Eating";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking";
    }
};
```

Now:

```
Dog d;
d.eat();
d.bark();
```

Dog inherited `eat()` from `Animal`.

Terminology

```
Animal
/
|
↓
Dog

Parent      Base
```

Animal:

- Base class
- Parent class
- Superclass

Dog:

- Derived class
- Child class
- Subclass

Why inheritance?

Primarily: **Code reusability**

Suppose:

```
Animal
├─ eat()
├─ sleep()
└─ breathe()
```

Instead of writing those functions again:

- Dog
- Cat
- Cow
- Horse

they can inherit them.

7. Types of Inheritance

Single inheritance

One parent → one child.

```
A
↓
B
```

```
class B : public A {};
```

Multilevel inheritance

```
A
↓
B
↓
C
```

Example:

```
class Animal {};
class Dog : public Animal {};
class Puppy : public Dog {};
```

Hierarchical inheritance

One parent → multiple children.

```
Animal
/      \
Dog      Cat
```

Multiple inheritance

Multiple parents → one child.

```
A      B
 \    /
  C
```

```
class C : public A, public B {};
```

Hybrid inheritance

Combination of two or more inheritance types.

8. Polymorphism

This is one of the highest-priority OOP topics.

Meaning

- **Poly** = Many
- **Morph** = Forms

Therefore:

Polymorphism means the same interface/name can exhibit different behavior depending on context.

Example:

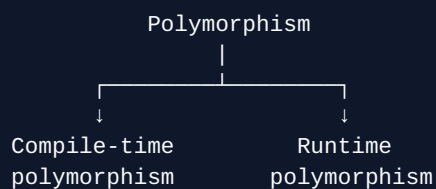
```
Animal
├── Dog → Bark
└── Cat → Meow
```

Both can have:

```
sound()
```

but behavior differs.

Two types



9. Function Overloading

Function overloading is compile-time polymorphism.

Same function name, but different parameter list.

```
class Calculator {
public:
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
};
```

We have:

- `add(int, int)`
- `add(int, int, int)`

The compiler determines which function to call.

Can we overload by return type only?

✗ No

This is invalid:

```
int add(int a, int b);  
double add(int a, int b);
```

Why? Because the function call:

```
add(10, 20);
```

doesn't provide enough information to distinguish based only on return type.

★ Accenture MCQ trap

Function overloading cannot be achieved by changing only the return type.

10. Method Overriding

Overriding occurs when a derived class provides its own implementation of a method inherited from the base class.

Example:

```
class Animal {  
public:  
    virtual void sound() {  
        cout << "Animal sound";  
    }  
};  
  
class Dog : public Animal {  
public:  
    void sound() override {  
        cout << "Bark";  
    }  
};
```

Here `Dog` overrides `Animal::sound()`.

Overloading vs Overriding

Feature	Overloading	Overriding
Polymorphism	Compile-time	Runtime
Relationship	Usually same class	Parent-child
Function name	Same	Same
Parameters	Different	Same/sufficiently matching signature

Feature	Overloading	Overriding
Purpose	Multiple ways to call operation	Replace inherited behavior
virtual	Not required	Important for runtime dispatch in C++

11. Constructor

A constructor initializes an object when it is created.

Example:

```
class Student {  
public:  
    Student() {  
        cout << "Student created";  
    }  
};
```

When:

```
Student s;
```

constructor automatically executes.

Constructor Properties

A constructor:

- has same name as class
- has no return type
- is called automatically
- initializes object
- can be overloaded

Types of Constructors

Default constructor

```
Student() {  
}
```

No parameters.

Parameterized constructor

```
Student(string n, int a) {  
    name = n;  
    age = a;  
}
```

Used to initialize with specific values.

Copy constructor

Creates an object from another object of the same class.

```
Student(const Student &s) {  
    name = s.name;  
    age = s.age;  
}
```

Example:

```
Student s1;  
Student s2 = s1;
```

12. Destructor

Destructor performs cleanup when an object is destroyed.

```
class Student {  
public:  
    ~Student() {  
        cout << "Object destroyed";  
    }  
};
```

Syntax:

```
~ClassName()
```

Constructor:

```
Student()
```

Destructor:

```
~Student()
```

Constructor vs Destructor

Constructor	Destructor
Initializes object	Cleans up object
Called during object creation	Called during object destruction
Same class name	~ + class name
Can be overloaded	Cannot be overloaded
Can take parameters	Cannot take parameters

13. Access Modifiers

C++ has:

- public
- private
- protected

Public

Accessible from anywhere where the object/member is accessible.

```
class A {
public:
    int x;
};

A obj;
obj.x = 10; // ✓
```

Private

Accessible only inside the class and permitted friends.

```
class A {
private:
    int x;
};

// Outside:
A obj;
obj.x = 10; // ✗
```

Protected

Accessible inside:

- the class
- derived classes
- friends

but not normally through an unrelated outside object.

```
class A {
protected:
    int x;
};
```

★ Access Modifier Comparison

Access Specifier	Same Class	Derived Class	Outside
public	✓	✓	✓
protected	✓	✓	✗
private	✓	✗*	✗

* A derived class cannot directly access a private base member.

● TIER 2

Now we move into the concepts that often distinguish a candidate who memorized OOP from one who actually understands it.

14. Virtual Function

A virtual function allows C++ to perform runtime polymorphism.

Example:

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal";
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Dog";
    }
};
```

Now:

```
Animal* ptr = new Dog();
ptr->sound();
```

Output:

```
Dog
```

Why? Because `sound()` is virtual.

Without virtual

```
class Animal {
public:
    void sound() {
        cout << "Animal";
    }
};
```

Then:

```
Animal* ptr = new Dog();
ptr->sound();
```

would resolve to the base implementation under ordinary non-virtual dispatch:

```
Animal
```

This is one of the most important C++ OOP traps.

15. Dynamic Binding

Binding means:

Connecting a function call to the function implementation that will execute.

Static/Early binding

Decision made at compile time.

Example:

```
obj.add();
```

for an overloaded function.

Dynamic/Late binding

Decision made at runtime.

Example:

```
Animal* ptr = new Dog();  
ptr->sound();
```

with `sound()` virtual.

Static vs Dynamic Binding

Static Binding	Dynamic Binding
Compile time	Runtime
Early binding	Late binding
Faster dispatch generally	Runtime dispatch
Overloading	Overriding through virtual dispatch

16. Pure Virtual Function

A pure virtual function has no required base implementation and makes the class abstract.

Syntax:

```
class Animal {  
public:  
    virtual void sound() = 0;  
};
```

The `= 0` means pure virtual.

17. Abstract Class

A class containing at least one pure virtual function is an abstract class.

Example:

```
class Animal {  
public:  
    virtual void sound() = 0;  
};
```

You cannot normally create an object of it:

```
Animal a; // ❌
```

But you can create a pointer/reference:

```
Animal* ptr;
```

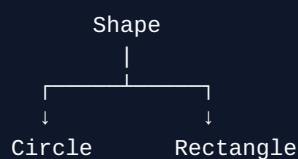
and point it to a derived object:

```
Dog d;  
ptr = &d;
```

Why abstract classes?

They provide a common interface/contract.

Example:



You can define:

```
virtual void draw() = 0;
```

Every concrete shape must provide its own implementation.

18. Interface

C++ doesn't have a dedicated `interface` keyword like Java.

But an interface-like abstraction can be created using an abstract class containing pure virtual functions.

Example:

```
class Payment {  
public:  
    virtual void pay() = 0;  
};  
  
class Razorpay : public Payment {  
public:  
    void pay() override {  
        cout << "Razorpay payment";  
    }  
};  
  
class Stripe : public Payment {  
public:  
    void pay() override {  
        cout << "Stripe payment";  
    }  
};
```

Both satisfy the same contract:

```
Payment
├──
├── Razorpay
└── Stripe
```

19. this Pointer

`this` is a pointer to the current object in a non-static member function.

Example:

```
class Student {
private:
    int age;
public:
    Student(int age) {
        this->age = age;
    }
};
```

Here:

`age` on the right is the parameter.

`this->age` means the object's member.

Why use this?

Mainly when parameter and member have the same name.

```
Student(int age) {
    this->age = age;
}
```

Think: `this` → current object

20. super Concept

This is especially important when comparing C++ with Java.

Java

`super` refers to the parent-class portion/member access.

Example:

```
class Dog extends Animal {
    void show() {
        super.sound();
    }
}
```

C++

There is no `super` keyword.

You explicitly use the base class:

```
Animal::sound();
```

Accenture trap

If the question is language-specific, don't automatically assume: `super = C++` .

C++ uses `BaseClass::member` instead.

21. Static Members

A static data member belongs to the class rather than each individual object.

Example:

```
class Student {  
public:  
    static int count;  
};  
int Student::count = 0;
```

There is only one shared count for the class.

```
Student  
├──  
│   ├── s1  
│   ├── s2  
│   └── s3  
  
All share:  
    count
```

Static Member Function

```
class Student {  
public:  
    static void show() {  
        cout << "Hello";  
    }  
};
```

Call:

```
Student::show();
```

No object is required.

Important restriction:

A static member function does not have a `this` pointer because it is not associated with a particular object.

Therefore it cannot directly access ordinary non-static instance members.

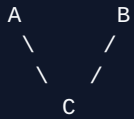
22. Multiple Inheritance

A class can inherit from more than one base class in C++.

```
class A {  
};  
class B {  
};
```

```
class C : public A, public B {
};
```

Structure:

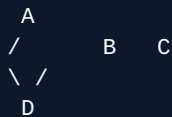


This is supported in C++.

23. Diamond Problem

This is an important Tier 2 concept.

Consider:

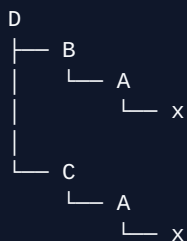


Both B and C inherit from A. Then D inherits from both.

Suppose:

```
class A {
public:
    int x;
};
```

Then:



D potentially contains two copies of A. So:

```
d.x
```

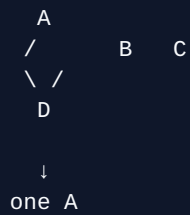
can become ambiguous.

Solution: Virtual Inheritance

```
class A {
public:
    int x;
};
class B : virtual public A {
};
class C : virtual public A {
```

```
};  
class D : public B, public C {  
};
```

Now only one shared A subobject exists within D.



Section A — Conceptual MCQs

Q1. Four Pillars of OOP

Which of the following is NOT one of the four fundamental pillars of OOP?

- A. Encapsulation
- B. Abstraction
- C. Inheritance
- D. Compilation

 **Answer: D. Compilation**

A ✗ Encapsulation — One of the four OOP pillars. It combines data/behavior and controls access to internal state.

B ✗ Abstraction — One of the four pillars. It hides unnecessary implementation details.

C ✗ Inheritance — One of the four pillars. It enables deriving new classes from existing classes.

D ✓ Compilation — Compilation is a language-processing step, not an OOP pillar.

Remember:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Q2. Encapsulation

A programmer declares a class variable balance as private and provides deposit() and getBalance() methods to access it.

Which OOP principle is primarily demonstrated?

- A. Inheritance
- B. Encapsulation
- C. Polymorphism
- D. Compilation

✓ **Answer: B. Encapsulation**

- A ✗ **Inheritance** — No parent-child relationship is involved.
- B ✓ **Encapsulation** — Internal data is protected and accessed through controlled methods.
- C ✗ **Polymorphism** — There are no multiple implementations/forms here.
- D ✗ **Compilation** — Compilation is unrelated to this design principle.

Q3. Abstraction

A user presses the start() button on a car without knowing how fuel injection, ignition, and engine control internally work.

Which principle is illustrated?

- A. Encapsulation
- B. Abstraction
- C. Inheritance
- D. Overloading

✓ **Answer: B. Abstraction**

- A ✗ **Encapsulation** — Encapsulation focuses on bundling and access control.
- B ✓ **Abstraction** — The user sees the necessary interface while implementation details remain hidden.
- C ✗ **Inheritance** — No parent-child relationship exists.
- D ✗ **Overloading** — No multiple methods with the same name are involved.

Q4. Code Reusability

Which OOP feature primarily promotes code reuse through a parent-child relationship?

- A. Encapsulation
- B. Abstraction
- C. Inheritance
- D. Constructor

☒ **Answer: C. Inheritance**

- A ☒ **Encapsulation** — Primarily concerns data/behavior bundling and access control.
- B ☒ **Abstraction** — Primarily hides implementation complexity.
- C ☒ **Inheritance** — A derived class can reuse members of a base class.
- D ☒ **Constructor** — Initializes objects; it isn't an inheritance mechanism.

Q5. Polymorphism

What does polymorphism literally represent in OOP?

- A. One class, one object
- B. One interface/name with multiple forms or behaviors
- C. Hiding all data
- D. Reusing only variables

☒ **Answer: B. One interface/name with multiple forms or behaviors**

- A ☒ — This describes neither the definition nor purpose of polymorphism.
- B ☒ — "Poly" means many and "morph" means forms.
- C ☒ — Data hiding is associated with encapsulation.
- D ☒ — Code reuse is more closely associated with inheritance.

Q6. Function Overloading

Which example represents function overloading?

A.

```
void add(int a, int b);  
void add(int a, int b, int c);
```

B.

```
class Dog : public Animal {};
```

C.

```
class Dog {  
    void bark();  
};
```

D.

```
private:  
    int age;
```

✓ **Answer: A**

A ✓ — Same function name with different parameter lists = overloading.

B ✗ — This is inheritance.

C ✗ — This merely declares a function.

D ✗ — This demonstrates access control/encapsulation.

Q7. Return Type and Overloading

Can two functions be overloaded by changing only their return type?

A. Yes

B. No

C. Only in C++

D. Only when functions are private

✓ **Answer: B. No**

Example:

```
int add(int a, int b);  
double add(int a, int b);
```

✗ Invalid as an overload set based solely on return type.

A ✗ — The compiler cannot select based only on the return type in a normal call.

B ✓ — Parameter list must differ.

C ✗ — C++ does not permit this.

D ✗ — Access modifier doesn't change this rule.

Q8. Method Overriding

A child class provides its own implementation of a method inherited from its parent. What is this called?

- A. Overloading
- B. Overriding
- C. Encapsulation
- D. Abstraction

✓ **Answer: B. Overriding**

- A ✗ **Overloading** — Same name but different parameter list.
- B ✓ **Overriding** — Derived class replaces inherited behavior.
- C ✗ **Encapsulation** — Access/data-control mechanism.
- D ✗ **Abstraction** — Hiding implementation details.

Q9. Compile-Time Polymorphism

Which is the most common example of compile-time polymorphism?

- A. Function overloading
- B. Virtual function overriding
- C. Dynamic binding
- D. Abstract class

✓ **Answer: A. Function overloading**

- A ✓ — Compiler selects the appropriate overloaded function.
- B ✗ — Virtual dispatch is associated with runtime polymorphism.
- C ✗ — Dynamic binding occurs at runtime.
- D ✗ — Abstract classes provide abstraction/contracts; they are not themselves a type of compile-time polymorphism.

Q10. Runtime Polymorphism

Which combination is most directly associated with runtime polymorphism in C++?

- A. Overloading + static binding
- B. Overriding + virtual function
- C. Constructor + destructor
- D. Encapsulation + inheritance

✓ **Answer: B. Overriding + virtual function**

- A ✗ — Overloading is compile-time polymorphism.
- B ✓ — A virtual function enables dynamic dispatch through a base pointer/reference.
- C ✗ — Constructors/destructors manage object lifetime.
- D ✗ — These are OOP concepts but don't define runtime polymorphism.

Q11. Constructor

Which statement about a C++ constructor is correct?

- A. It must return void
- B. It has the same name as the class
- C. It must be declared static
- D. It is called manually before every object access

✓ **Answer: B. It has the same name as the class**

- A ✗ — Constructors have no return type.
- B ✓ — Constructor name matches the class name.
- C ✗ — Constructors are not static.
- D ✗ — Constructors are automatically invoked during object initialization.

Q12. Destructor

Which syntax represents a destructor for class Student?

- A. Student()
- B. delete Student()
- C. ~Student()
- D. destructor Student()

✓ **Answer: C. ~Student()**

- A ✗ — Constructor.
- B ✗ — Not destructor syntax.
- C ✓ — ~ followed by class name defines a destructor.
- D ✗ — Not valid C++ syntax.

Q13. Access Modifiers

Which access modifier allows access from anywhere the object/member is accessible?

- A. private
- B. protected
- C. public
- D. internal

✓ **Answer: C. public**

- A ✗ — Private members are restricted to the class/friends.
- B ✗ — Protected members are accessible in the class and derived classes/friends.
- C ✓ — Public members can be accessed from outside the class.
- D ✗ — internal is not a C++ access specifier.

Q14. Protected Member

Which statement about a protected member in C++ is correct?

- A. It can be accessed by unrelated external code through an object
- B. It can be accessed by derived classes
- C. It can never be accessed by a child class
- D. It is exactly the same as private

✓ **Answer: B. It can be accessed by derived classes**

- A ✗ — Normal external access isn't permitted.
- B ✓ — Derived classes can access inherited protected members subject to the language's access rules.
- C ✗ — This is the opposite of the actual behavior.
- D ✗ — private and protected differ specifically in derived-class access.

Q15. Virtual Function

What is the primary purpose of a virtual function in C++?

- A. Compile-time function overloading
- B. Runtime polymorphism
- C. Memory allocation
- D. Data encryption

✓ **Answer: B. Runtime polymorphism**

- A ✗ — Overloading is compile-time polymorphism.
- B ✓ — Virtual functions enable dynamic dispatch.
- C ✗ — They don't primarily perform memory allocation.
- D ✗ — They have nothing to do with encryption.

 Section B — Tier 2 MCQs**Q16. Pure Virtual Function**

Which declaration represents a pure virtual function?

- A. `void display();`
- B. `virtual void display();`
- C. `virtual void display() = 0;`
- D. `pure virtual display();`

✓ **Answer: C**

- A ✗ — Ordinary function declaration.
- B ✗ — Virtual function, but not pure virtual.
- C ✓ — `= 0` makes it pure virtual.
- D ✗ — Invalid C++ syntax.

Q17. Abstract Class

Consider:

```
class Animal {  
public:  
    virtual void sound() = 0;  
};
```

What is Animal?

- A. Concrete class
- B. Abstract class
- C. Static class
- D. Friend class

✓ **Answer: B. Abstract class**

- A ✗ — A class with a pure virtual function is not concrete.
- B ✓ — It cannot be directly instantiated.
- C ✗ — It isn't a static class.
- D ✗ — No friend relationship exists.

Q18. Can an Abstract Class Have a Constructor?

- A. No
- B. Yes
- C. Only if it has no virtual functions
- D. Only in Java

✓ **Answer: B. Yes**

An abstract class can have constructors.

```
class Animal {  
public:  
    Animal() {  
        cout << "Animal";  
    }  
  
    virtual void sound() = 0;  
};
```

- A ✗ — Abstract classes can have constructors.
- B ✓ — The constructor initializes the base-class portion of derived objects.
- C ✗ — It can have pure virtual functions and still have a constructor.
- D ✗ — C++ supports this.

Q19. Static Binding

Static binding occurs at:

- A. Runtime
- B. Compile time
- C. Linker shutdown
- D. Object destruction

✓ **Answer: B. Compile time**

- A ✗ — Runtime binding occurs during execution.
- B ✓ — Static/early binding is resolved at compile time.
- C ✗ — Not the definition of static binding.
- D ✗ — Destruction is unrelated.

Q20. Dynamic Binding

Dynamic binding is also called:

- A. Early binding
- B. Compile-time binding
- C. Late binding
- D. Static binding

✓ **Answer: C. Late binding**

- A ✗ — Early binding = static binding.
- B ✗ — Compile-time binding is static binding.
- C ✓ — Dynamic binding is also called late binding.
- D ✗ — Static and dynamic binding are different concepts.

Q21. this Pointer

In C++, what does this refer to?

- A. Parent class
- B. Current object
- C. Current class definition itself
- D. Previous object

✓ **Answer: B. Current object**

- A ✗ — Base-class access is handled differently.
- B ✓ — this points to the current object in a non-static member function.
- C ✗ — It represents the current object, not the class as a type.
- D ✗ — No such meaning.

Q22. Static Member Function

Which statement is correct about a static member function?

- A. It always requires an object to call it
- B. It has a this pointer
- C. It can be called using the class name
- D. It can directly access every non-static member

✓ **Answer: C**

Example:

```
Student::show();
```

- A ✗ — An object is not required.
- B ✗ — Static member functions don't have a this pointer.
- C ✓ — Class-name qualification can be used.
- D ✗ — It cannot directly access ordinary instance members without an object.

Q23. Multiple Inheritance

Which language directly supports multiple inheritance of classes?

- A. C++
- B. Java classes
- C. C# classes
- D. HTML

✓ **Answer: A. C++**

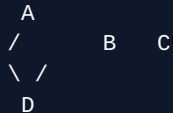
- A ✓ — C++ supports multiple base classes.
- B ✗ — Java doesn't support multiple inheritance of classes, though it supports multiple interfaces.
- C ✗ — C# doesn't support multiple class inheritance, though it supports multiple interfaces.
- D ✗ — HTML isn't an OOP language.

Q24. Diamond Problem

The diamond problem primarily occurs because:

- A. A class has no constructor
- B. Multiple inheritance creates multiple paths to the same base class
- C. Functions are overloaded
- D. A class contains private variables

✓ **Answer: B**



- A ✗ — Constructors aren't the cause.
- B ✓ — D reaches A through both B and C.
- C ✗ — Function overloading isn't the cause.
- D ✗ — Private data isn't responsible.

Q25. Diamond Problem Solution

Which C++ mechanism can solve the classic diamond inheritance problem?

- A. Function overloading
- B. Encapsulation
- C. Virtual inheritance
- D. Constructor overloading

✓ **Answer: C. Virtual inheritance**

- A ✗ — Doesn't address duplicate base subobjects.
- B ✗ — Doesn't solve inheritance ambiguity.
- C ✓ — Virtual inheritance allows a shared base-class subobject.
- D ✗ — Constructor overloading doesn't solve the hierarchy issue.

 Section C — Scenario-Based MCQs

These are particularly useful because Accenture can test whether you can identify the concept from a situation rather than from a definition.

Q26. Banking System

A banking application does not allow users to directly modify balance. Instead, users must use `deposit()` and `withdraw()` methods that validate transactions.

Which OOP principle is primarily being used?

- A. Inheritance
- B. Encapsulation
- C. Polymorphism
- D. Multiple inheritance

✓ **Answer: B. Encapsulation**

- A ✗ — No parent-child relationship is involved.
- B ✓ — Internal state is protected and modified through controlled methods.
- C ✗ — No multiple forms of behavior are involved.
- D ✗ — No multiple base classes are involved.

Q27. Payment System

A payment application has:

```
pay()
```

The same operation works differently for:

- CreditCard
- UPI
- PayPal

Which OOP concept best describes this design?

- A. Polymorphism
- B. Encapsulation only
- C. Constructor chaining
- D. Data hiding only

✓ **Answer: A. Polymorphism**

- A ✓ — Same interface, different implementations.
- B ✗ — Encapsulation may exist, but it doesn't describe the varying behavior.
- C ✗ — Constructors aren't the focus.
- D ✗ — Data hiding isn't the main concept here.

Q28. Employee System

A company has:

```
Employee
  ↓
Manager
```

Manager automatically gets common employee functionality such as `calculateSalary()`.

Which concept is being used?

- A. Abstraction
- B. Inheritance
- C. Overloading
- D. Dynamic binding

✓ **Answer: B. Inheritance**

- A ✗ — Hiding complexity isn't the central point.
- B ✓ — Manager derives/reuses functionality from Employee.
- C ✗ — No same-name/different-parameter methods.
- D ✗ — No runtime dispatch is described.

Q29. Vehicle System

A program has:

```
void start(int key);
void start(string fingerprint);
```

Which concept is demonstrated?

- A. Method overriding
- B. Function overloading
- C. Dynamic binding
- D. Inheritance

✓ **Answer: B. Function overloading**

- A ✗ — No parent-child relationship or replacement of inherited method.
- B ✓ — Same method name with different parameter types.
- C ✗ — This is compile-time selection.
- D ✗ — No inheritance shown.

Q30. Animal Application

Consider:

```
class Animal {  
public:  
    virtual void sound() {  
        cout << "Animal";  
    }  
};  
  
class Dog : public Animal {  
public:  
    void sound() override {  
        cout << "Bark";  
    }  
};
```

Then:

```
Animal* a = new Dog();  
a->sound();
```

What is printed?

- A. Animal
- B. Dog
- C. Bark
- D. Compilation error

 **Answer: C. Bark**

- A**  — This would be the expected result for non-virtual dispatch in this setup.
- B**  — Dog is a class name, not what sound() prints.
- C**  — sound() is virtual, so runtime dispatch selects Dog::sound().
- D**  — The code is valid assuming the necessary declarations/includes.

Q31. Abstract Payment System

A company wants every payment method to implement:

```
pay()
```

but does not want anyone to instantiate the generic Payment class.

What is the best design?

- A. Concrete class with no methods
- B. Abstract class with a pure virtual pay()
- C. Private constructor only
- D. Function overloading

✓ **Answer: B**

Example:

```
class Payment {  
public:  
    virtual void pay() = 0;  
};
```

- A ✗ — Doesn't enforce implementation.
- B ✓ — Provides a contract and prevents direct instantiation.
- C ✗ — A private constructor isn't the standard mechanism for an abstract interface contract.
- D ✗ — Overloading doesn't enforce derived-class implementation.

Q32. Student Constructor

Consider:

```
class Student {  
    int age;  
public:  
    Student(int age) {  
        this->age = age;  
    }  
};
```

Why is `this->age` used?

- A. To access the parent class's age
- B. To access the current object's member age
- C. To create a new object
- D. To make age static

 **Answer: B**

- A  — this refers to the current object.
- B  — `this->age` identifies the object's member.
- C  — this doesn't create an object.
- D  — It has nothing to do with static members.

Q33. Shared Counter

Suppose a university wants to maintain one counter representing the total number of Student objects created, shared across all objects.

Which should be used?

- A. Instance variable
- B. Static data member
- C. Local variable
- D. Pure virtual function

 **Answer: B. Static data member**

- A  — Each object would have its own separate copy.
- B  — Static data members are shared by the class's objects.
- C  — A local variable exists only within its scope.
- D  — Pure virtual functions are unrelated.

Q34. Diamond Inheritance Scenario

A developer creates:

```
graph TD
    Person
    Student --> Person
    Employee --> Person
    Intern --> Student
    Intern --> Employee
```

Both Student and Employee inherit from Person.

Intern therefore gets two paths to Person.

What problem may occur?

- A. Encapsulation problem
- B. Diamond problem
- C. Constructor overloading
- D. Function overloading

✓ **Answer: B. Diamond problem**

- A ✗ — Not the specific issue.
- B ✓ — Multiple paths to the same base class create the classic diamond.
- C ✗ — Constructors aren't the source of the hierarchy ambiguity.
- D ✗ — Overloading isn't involved.

Q35. Interface Scenario

A software system requires every notification service to implement:

```
send()
```

There are:

- EmailNotification
- SMSNotification
- PushNotification

Which design best represents this requirement?

- A. Interface/abstract contract
- B. Only function overloading
- C. Only encapsulation
- D. Destructor

✓ **Answer: A. Interface/abstract contract**

- A ✓ — A common contract lets each implementation provide its own send().
- B ✗ — Overloading changes parameter lists; it doesn't establish implementation contracts.
- C ✗ — Encapsulation doesn't define this common behavioral contract.
- D ✗ — Destructor concerns object lifetime.